

TECHNICAL DESIGN BRIEF

# Camera AgentSQL

Agent architecture, tool and model choices, and the reliability case for a constrained natural-language database agent.

The core decision: use the LLM to understand language—not to own policy, state, query construction, or database access.

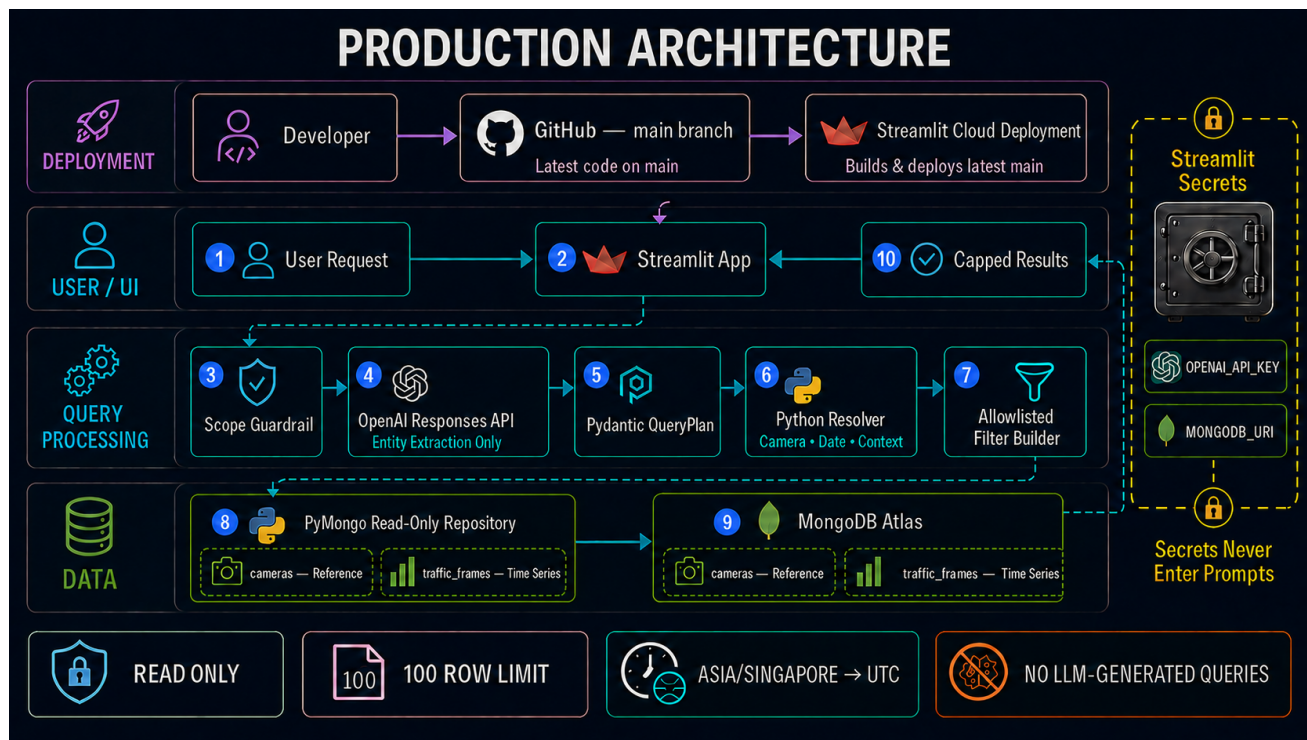
Prepared by Ritik Srivastava

Traffic-camera query agent | Python 3.12 | September 2026

# 1 Architecture at a glance

## A narrow model boundary, deterministic execution

Camera AgentSQL converts a natural-language request into a validated query plan, then hands control to ordinary Python. The model never emits executable MongoDB syntax. Every consequential operation—camera resolution, date arithmetic, context merging, filter construction, validation, and execution—remains explicit and testable.



Production flow. Solid arrows remain within a layer; dashed arrows cross trust or execution boundaries.

This is intentionally not an autonomous tool loop. One typed extraction call is enough for the bounded problem; adding an agent framework would increase failure modes without improving control.

## 2 Request-to-result path

### Ten visible steps; three hard trust boundaries

#### 1–2 · Intake

Streamlit receives the request and maintains session state. Conservative typo normalization corrects known query terms without rewriting camera names.

#### 3–5 · Interpret

Pre-query guardrails reject unsafe intent. The Responses API extracts a strict Pydantic QueryPlan; unknown properties are forbidden.

#### 6–7 · Resolve

Python resolves cameras, Singapore-local dates, weekdays, overnight windows, follow-up context, sort order, and limit.

#### 8–10 · Execute

An allowlisted filter reaches one read-only repository method over `traffic_frames`. MongoDB results are sorted, capped, display-sanitized, and returned to Streamlit.

*The model is deliberately boxed into entity and intent extraction.*



### Data contract

The cameras collection is reference metadata: `camera_id`, canonical `camera_name`, acronym, aliases, and active state. `traffic_frames` is a native time-series collection keyed by `captured_at` with `camera_name` metadata, `frame_id`, and `frame_img_url`. The synthetic sample contains 14,640 hourly frame records from 1 August through 30 September 2026.

### 3 Tool and model choices

Choose the smallest component that can enforce each responsibility

| Choice               | Why it fits                                                                                        | Boundary                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| OpenAI Responses API | One structured extraction call maps natural language to QueryPlan.                                 | No query syntax, tool selection, or direct database access.                         |
| gpt-5.4-mini         | Default model balances instruction following, structured output, latency, and cost for extraction. | Replaceable through OPENAI_MODEL; downstream controls do not depend on model trust. |
| Pydantic             | A strict typed contract rejects extra fields and invalid date/limit shapes.                        | Validation proves shape—not business correctness—so Python still resolves values.   |
| RapidFuzz            | Local matching covers acronyms, aliases, and reasonable spelling errors.                           | Low-confidence or close competing matches trigger clarification.                    |
| PyMongo + Atlas      | Explicit find operations fit MongoDB time-series data and remain easy to audit.                    | No generic command or write interface; hard maximum of 100 records.                 |
| Streamlit            | Fast multipage review UI, chat state, trace display, and deployment from GitHub main.              | Presentation layer only; it does not weaken repository controls.                    |
| zoneinfo + pytest    | Standard-library timezone conversion plus deterministic tests for boundary behavior.               | Asia/Singapore input is converted to half-open UTC intervals.                       |

**Reliability comes from separation of duties: probabilistic interpretation at the edge, deterministic enforcement at the core.**



## 4 Guardrails and query validation

The database never sees raw user instructions

### Before the model

- Reject insert, update, delete, drop, truncate, upsert, schema changes, arbitrary MongoDB commands, secret disclosure, and rule-override attempts.
- Reject empty or oversized requests before API or database work.



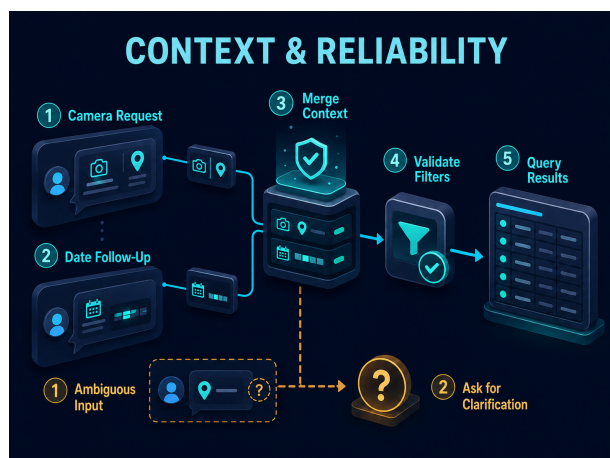
### Validation layers

- QueryPlan permits only known fields and constrained enums; result\_limit cannot exceed 100.
- Camera terms must resolve to active canonical records. Ambiguous matches return suggestions instead of guessing.
- Dates resolve in Asia/Singapore, then convert to UTC half-open bounds using \$gte and \$lt. Overnight ranges advance the end date deterministically.
- Mongo filters accept only camera\_name and captured\_at, and only \$and, \$or, \$in, \$gte, and \$lt.
- The user query path exposes only traffic\_frames.find(...). Database credentials are separately restricted to read permissions.
- Authentication, rate-limit, timeout, malformed-output, resolution, empty-result, and database failures become bounded, secret-safe messages.

**Injection resistance does not depend on recognizing every malicious phrase. Even if text passes the first check, it cannot become executable syntax or escape the repository allowlist.**

## 5 Context, ambiguity, and time

### Preserve filters—not an unrestricted transcript



### Follow-up policy

Session state stores canonical camera names, a validated date window, weekdays, and optional start/end times. Clear follow-ups inherit omitted filters; explicit new values replace the corresponding prior field. Fresh commands do not silently inherit an old camera.

### Example

1. “Show me frames from CTE.”
2. “How about only those from this week?”

The second turn keeps Central Expressway and adds the current-week date constraint.

### Ambiguity policy

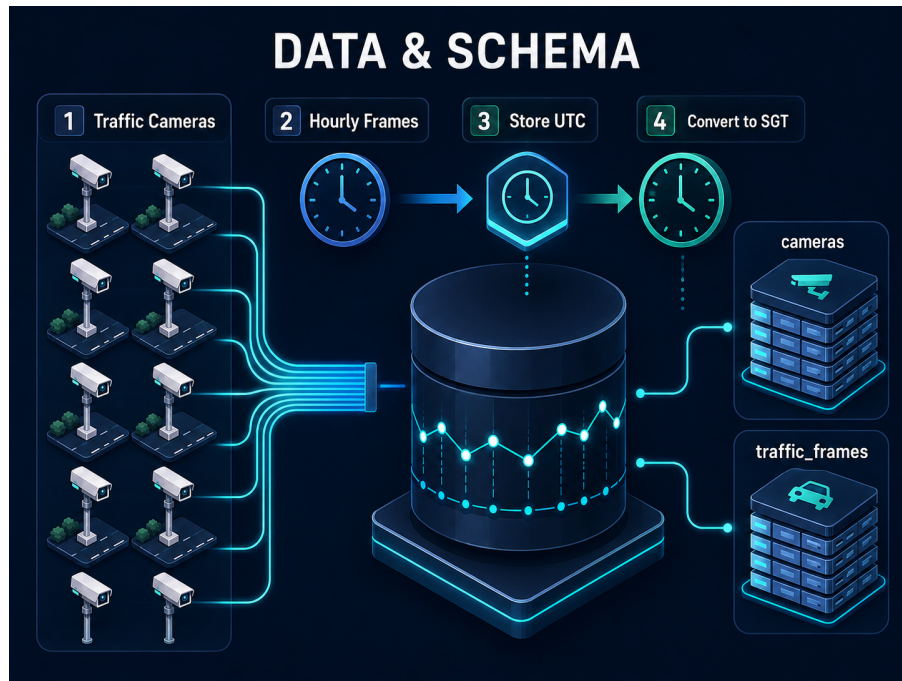
Low-confidence camera text or competing matches returns clarification suggestions. The system prefers a visible non-answer over a confident query against the wrong camera.

### Temporal semantics

A session-wide Singapore date anchors “today,” “yesterday,” “this week,” and relative month ranges such as “15th to 18th of last month.” The synthetic dataset extends from 1 August to 30 September 2026 so relative-date queries remain testable later in September. Future rows are test fixtures, not forecasts: explicit dates after the session date are rejected, and broad queries are automatically capped at today. Calendar validity is checked before execution.

## 6 Why this architecture is robust

Control increases as the request approaches data



*Reference metadata supports resolution; only time-series frame records are queryable through chat.*

### Design judgment

- One model call keeps prompts short, latency bounded, and failures attributable.
- Typed plans make model output inspectable without exposing hidden reasoning.
- Deterministic Python makes dates, context, future-date rejection, filters, and limits unit-testable.
- A repository allowlist provides a final technical boundary independent of prompt quality.
- The Query Processing UI reveals normalized input, resolved values, safe filter, sort, limit, and row count—enough to audit behavior without leaking prompts or credentials.

**The trade-off is deliberate: less autonomy, more predictability. For a read-only traffic-camera retrieval task, that is the correct engineering choice.**

### Verification focus

The test suite covers strict validation, malformed plans, typo and alias resolution, ambiguity, relative and ranged dates, weekdays, overnight windows, Singapore-to-UTC conversion, follow-up inheritance and reset, unsafe requests, filter allowlists, hard limits, and mocked API/database failures.

Implementation basis: agents.md, docs/project\_context.md, and the current Camera AgentSQL source on feature-v2.